

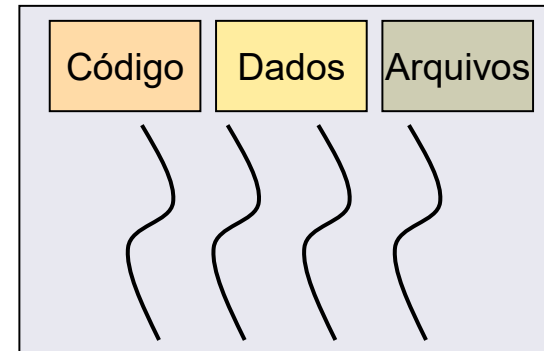
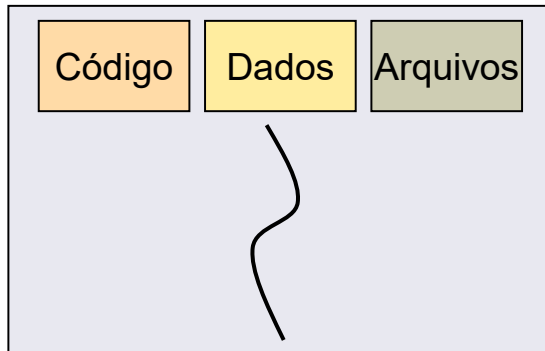
Sistemas Operacionais

Threads

Threads

- Processo
 - Um único fluxo de execução
 - Espaço de endereçamento, descritores de arquivos abertos, permissões de acesso, quotas...
- Threads
 - Múltiplos fluxos de execução
 - Multithreading -> mesmo espaço de endereçamento

Threads



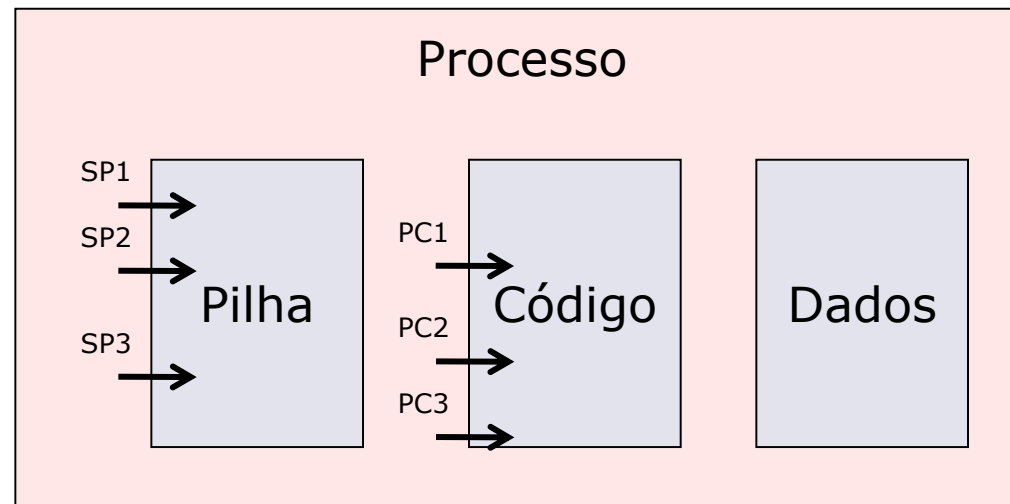
Multithreading: Associar vários fluxos de execução a um único processo

Threads

- Um fluxo de instrução é implementado através do Contador de Programa (PC) e do Stack Pointer (SP)
- Estruturas comuns compartilhadas entre os threads
 - Código
 - Dados
 - Descritor de processo

Vários Fluxos de Execução (threads)

- Vários fluxos de execução em um único processo



Threads

- Processo leve
- Unidade básica de utilização da CPU
- Existem no interior de um processo

Descritor de thread

- ▣ ID Thread
- ▣ Prioridade
- ▣ Contador de Programa
- ▣ Registradores
- ▣ Pilha

▣ Compartilha com *Threads* do mesmo processo

- Espaço de endereçamento (Seção de dados, Seção de código, arquivos abertos, recursos do SO...)
- Gerência é mais leve
- Chaveamento de contexto mais rápido

Threads

- Aplicações em PCs são multithreading
 - Thread único
 - Um cliente de cada vez
- Multithread
 - Aceita requisições de vários clientes
- Navegador Web
- Servidor Web

Benefícios do Uso de Threads

- Capacidade de Resposta
 - Navegador Web
- Compartilhamento de Recursos
 - Mesmo espaço de endereço do processo
 - Memória, recursos do processo que pertence
- Economia
 - Troca de contexto de threads (alocação de memória e recursos) x processos
- Aplicações multiprocessador
 - Paralelismo

Threads de Usuário

- Implementadas através de biblioteca ligada ao programa do usuário (Threads de usuário ou do nível do usuário)
- Visíveis ao programador e desconhecidos do Kernel
- Gerenciamento (criação/escalonamento) através de bibliotecas de threads
 - Pthreads Posix
 - C-threads Mach
 - threads Solaris
- Gerenciamento mais rápido (Kernel não envolvido)

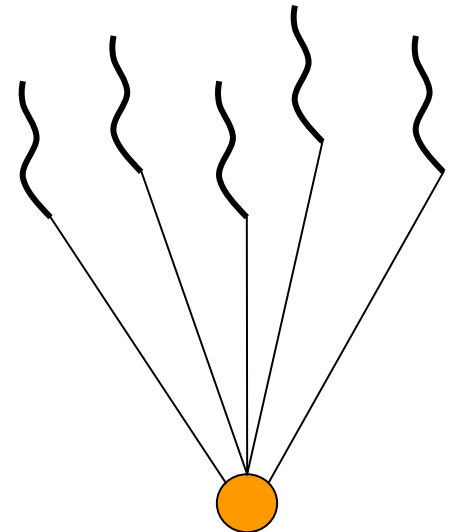
Threads de Kernel

- O SO suporta diretamente o conceito de Thread
 - A gerência do SO é voltada ao conceito de thread
- Gerenciamento através do Kernel
- São escalonados para execução nos vários processadores
- Mais lentos
- Suportadas diretamente pelo SO

Os SO fornecem suporte a *threads* de usuário e de kernel

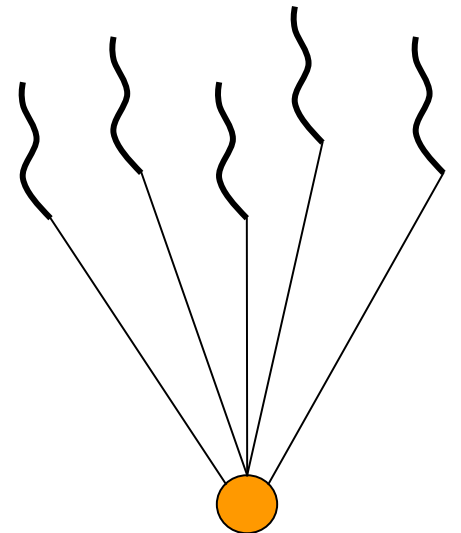
Modelos de *Multithreading*

- N:1 Muitos-para-um (*many-to-one*)
- Gerenciamento de threads é feito no espaço do usuário (aplicação)
 - São implementadas no espaço do usuário (uso de biblioteca ligada ao programa)
 - API para uso de threads
- Chaveamento de Contexto mais rápido
- Menor custo para criação/destruição
- O SO divide o tempo entre os processos
 - São manipuladas através de um único fluxo de instrução pertencente a um processo



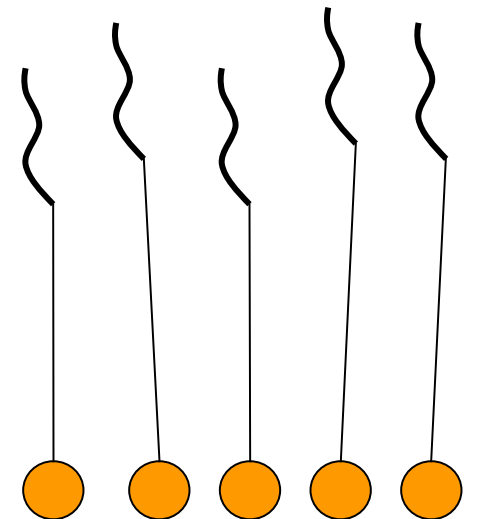
Modelos de *Multithreading*

- A troca de contexto entre threads é feita no modo usuário pelo escalonador embutido na biblioteca
- Se um thread está efetuando E/S, acontece o bloqueio de todas as threads do sistema
- Não é possível utilizar multiprocessadores
- Um thread é escalonado de cada vez



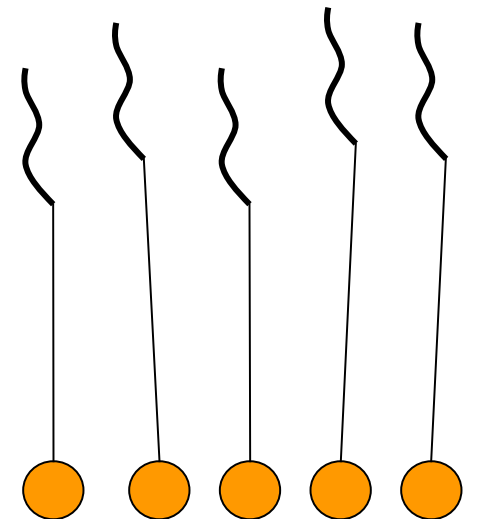
Modelos de *Multithreading*

- 1:1 Um-para-um (one-to-one)
 - Maior concorrência
 - Threads a nível do sistema
 - O sistema operacional mantém informações sobre processos e threads
 - Maior custo de gerenciamento
 - Exigem chamadas ao SO
 - Troca de contexto exige a intervenção do SO



Modelos de *Multithreading*

- É possível a execução paralela (ambiente multiprocessado)
- Permite sobrepor operações de processamento com I/O
- Desempenho da aplicação
- Depende número de threads suportados pelo sistema
- Win NT, OS/2

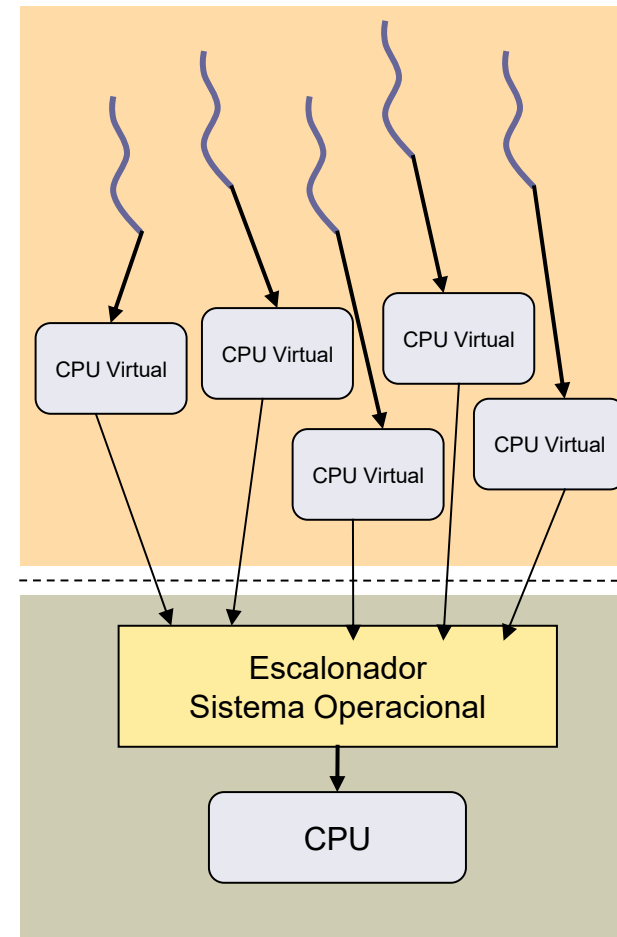


Modelos de *Multithreading*

- 1:1 Um-para-um (*one-to-one*)

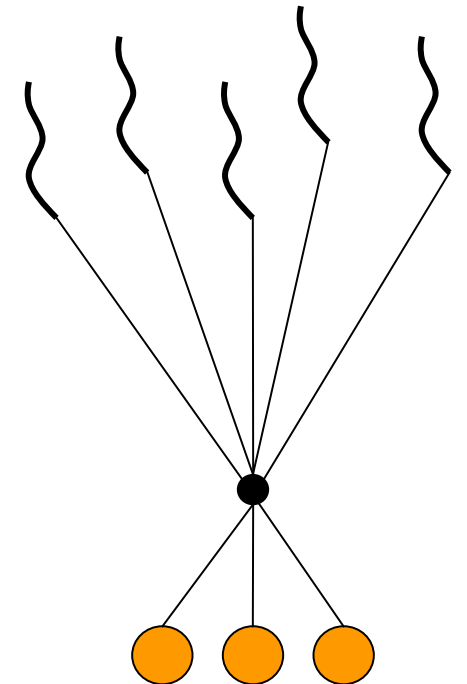
Espaço do Usuário

Espaço do Sistema



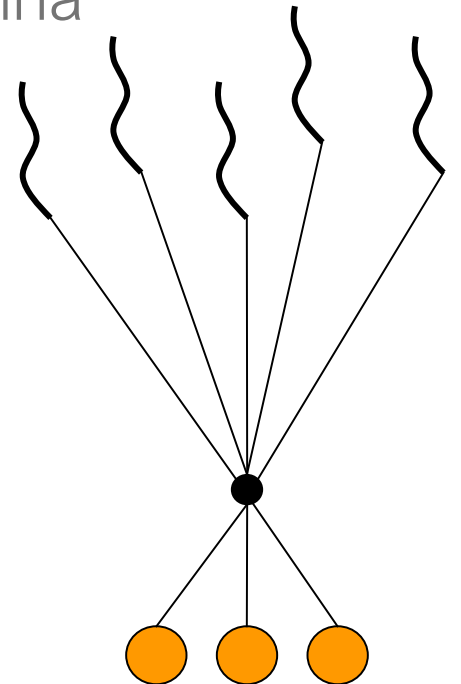
Modelos de *Multithreading*

- M:N Muitos-para-muitos (many-to-many)
 - Combina dos modelos N:1 e 1:1
 - M threads do usuário
 - N threads do SO
 - Multiplexa threads de usuários em um número menor ou igual de threads de kernel
 - Escalonamento de 2 níveis
 - Nível de usuário
 - Nível de sistema



Modelos de *Multithreading*

- M:N Muitos-para-muitos (*many-to-many*)
 - É possível criar tantos threads quantos forem necessários
 - Há um número de threads específico por aplicação ou máquina
 - Bloqueio -> escalonar outra para execução
 - Há paralelismo

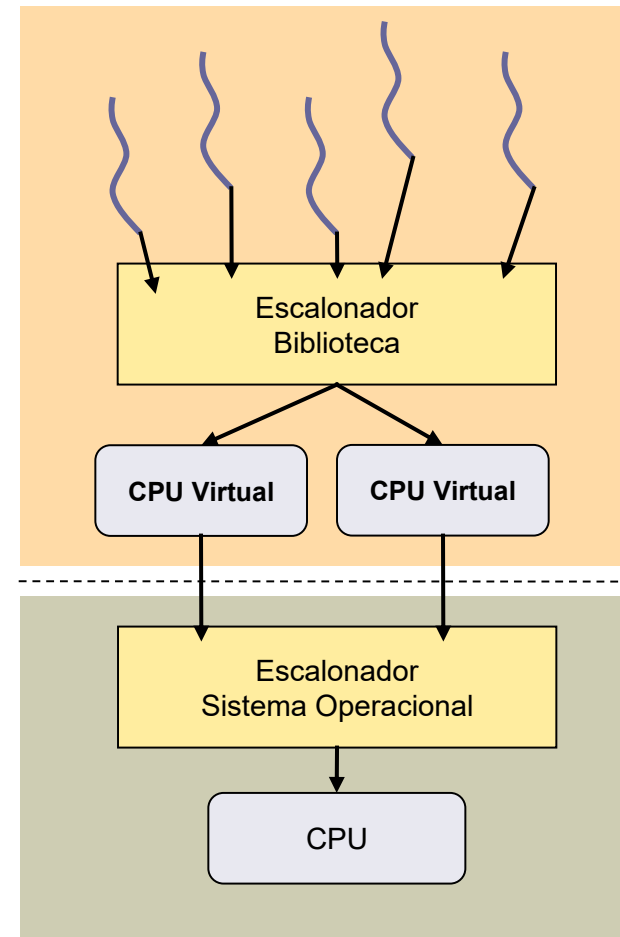


Modelos de *Multithreading*

- M:N Muitos-para-muitos (*many-to-many*)

Espaço do Usuário

Espaço do Sistema



Vantagens de *Multithreading*

- Threads x Processos
 - Tempo de criação/destruição
 - Troca de contexto
- Threads compartilham o mesmo espaço de endereçamento do processo que as criou, assim é possível o compartilhamento de memória sem a interação com o núcleo do sistema.

Sistemas Operacionais

Threads Java

Introdução

- Pode-se especificar fluxos de execução (*threads*), designando uma parte de um programa que pode ser executado simultaneamente com outras *threads*.
- *Multithreading* é parte da própria linguagem (pacote java.lang)
- Cada thread tem uma prioridade

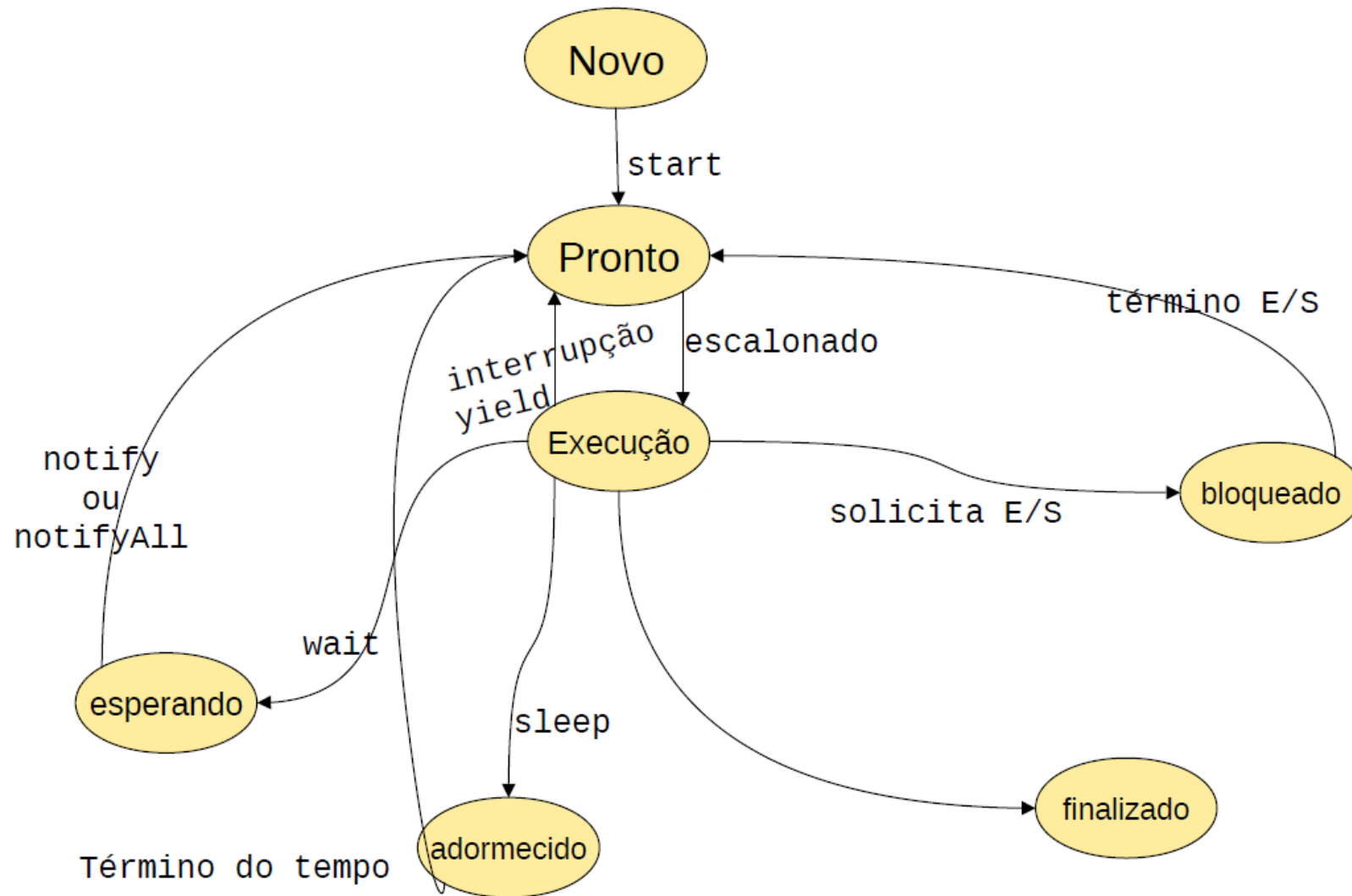
Threads em Java

- Há duas formas para uso de threads:
 - Criar uma subclasse da classe *Thread* (herança) e reescrever o método *run()*
 - A classe *Thread* implementa a interface *Runnable*
 - O método *run* é o “corpo” da *Thread*
 - Criar uma classe que implementa a interface
 - *Runnable*

Classe Thread

- Alguns métodos
 - Construtores
 - run()
 - start()
 - sleep()
 - join()

Estados de Threads



Exemplo 1

```
3 class Principal extends Thread {
4     public Principal(String str) {
5         super(str);
6     }
7
8     public void run() {
9         for (int i = 0; i < 30; i++) {
10             System.out.println(i + " " + getName());
11         }
12         System.out.println(getName() + " TERMINOU!");
13     }
14
15     public static void main(String args[]) {
16         Principal Thread1 = new Principal("Primeira");
17         Principal Thread2 = new Principal("Segunda");
18         Thread1.start();
19         Thread2.start();
20         for (int i = 0; i < 20; i++)
21             System.out.println(i + "Thread Principal");
22         System.out.println("THREAD PRINCIPAL - Término da Execução");
23     }
24 }
```

Exemplo 2

```
19 class PrintThread extends Thread {
20     private int sleepTime;
21
22     public PrintThread(String str) {
23         super(str);
24         sleepTime = (int) (Math.random() * 5000);
25         System.out.println(getName() + " dormirá " + sleepTime + "ms...");
26     }
27
28     public void run() {
29         try {
30             System.out.println(getName() + " indo dormir.");
31             Thread.sleep(sleepTime);
32         } catch (InterruptedException e) {
33             System.out.println(e.toString());
34         }
35         System.out.println(getName() + " TERMINOU!");
36     }
37 }
```

Exemplo 2 (Continuação)

```
3 public class ThreadTester {
4     public static void main(String args[]) {
5         PrintThread thread1, thread2, thread3, thread4;
6         thread1 = new PrintThread("thread1");
7         thread2 = new PrintThread("thread2");
8         thread3 = new PrintThread("thread3");
9         thread4 = new PrintThread("thread4");
10        System.out.println("\nIniciando threads...");
11        thread1.start();
12        thread2.start();
13        thread3.start();
14        thread4.start();
15        System.out.println("Threads iniciadas");
16    }
17 }
```

Exemplo 3

```
3 public class Mensagem {
4     private boolean pronta = false;
5
6     public synchronized void esperarMensagem() {
7         while (!pronta) {
8             try {
9                 System.out.println("Thread esperando mensagem...");
10                wait(); // espera até alguém chamar notify
11            } catch (InterruptedException e) {
12                e.printStackTrace();
13            }
14        }
15        System.out.println("Mensagem recebida!");
16    }
17
18    public synchronized void enviarMensagem() {
19        System.out.println("Preparando mensagem...");
20        pronta = true;
21        notify(); // acorda a thread que está esperando
22        System.out.println("Mensagem enviada!");
23    }
24 }
```

Exemplo 3 (continuação)

```
3 public class Principal {
4     public static void main(String[] args) {
5
6         Mensagem msg = new Mensagem();
7
8         Thread t1 = new Thread(() -> {
9             msg.esperarResultado();
10        });
11
12        Thread t2 = new Thread(() -> {
13            try {
14                Thread.sleep(2000);
15            } catch (InterruptedException e) {
16                e.printStackTrace();
17            }
18            msg.enviarResultado();
19        });
20
21        t1.start();
22        t2.start();
23    }
24 }
```

Sistemas Operacionais

POSIX Threads

POSIX Threads

- POSIX Threads, ou Pthreads, é um padrão estabelecido pela IEEE em 1995 que define uma interface padronizada para a criação e gerenciamento de threads.
- Esse padrão é amplamente utilizado em sistemas operacionais baseados em Unix e Linux, permitindo a execução concorrente de múltiplos fluxos de execução dentro de um mesmo processo.
- Para utilizar Pthreads em programas escritos em linguagem C, é necessário incluir a biblioteca por meio da diretiva `#include <pthread.h>`.

POSIX Threads

- As principais funções da biblioteca estão relacionadas à criação, ao término e à sincronização de threads, destacando-se o uso de `pthread_create`, `pthread_exit` e `pthread_join`.
- A seguir, será apresentado um exemplo prático de criação e manipulação de threads utilizando Pthreads.

POSIX Threads - Criação de Threads

- A função `pthread_create` é responsável por criar uma nova thread, permitindo a execução concorrente dentro do programa.
- Seu protótipo recebe quatro parâmetros: o identificador da thread, os atributos (que podem ser definidos como padrão com `NULL`), a função que será executada pela thread e o argumento que será passado para essa função.
- `#include <pthread.h>`
 - `int pthread_create (pthread_t * thread, pthread_attr_t * attr, void * (*start_routine)(void *), void *arg);`

POSIX Threads - Término de Threads

- A função `pthread_exit` é utilizada para finalizar a execução de uma thread, podendo retornar um valor que posteriormente pode ser recuperado por outra thread.
- Por fim, a função `pthread_join` é responsável por fazer com que uma thread aguarde o término de outra, garantindo sincronização entre elas; caso haja um valor de retorno, este pode ser armazenado para uso posterior.
- Essas funções são fundamentais para o controle do ciclo de vida e da coordenação entre threads em aplicações concorrentes.
- `#include <stdlib.h>`
 - `void pthread_exit (void *retval);`

POSIX Threads - Espera

- A função `pthread_join` suspende a execução da thread que a chamou até que a thread especificada como argumento seja finalizada.
- Esse mecanismo é utilizado para sincronizar a execução entre threads, garantindo que uma só continue após a conclusão da outra.
- Caso a thread finalizada retorne algum valor diferente de nulo, esse valor pode ser recuperado e armazenado por meio do parâmetro `thread_return`.
- `#include <stdlib.h>`
 - `int pthread_join (pthread_t th, void **thread_return);`

POSIX Threads

- Ver códigos na MinhaUFN

Referências Bibliográficas

- SILBERSCHATZ, Abraham; GALVIN, Peter; GAGNE, Greg. Sistemas Operacionais: com Java. Rio de Janeiro: Elsevier, 2004.
- OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. T. Sistemas Operacionais. Porto Alegre: Sagra-Luzzatto, 2000
- DEITEL, H. M. Java como Programar. Porto Alegre: Bookman, 2003.